Custom Agents

ragent supports custom agent definitions in two formats:

1. **Agent Profiles** (`.md`) — Markdown files with JSON frontmatter. The markdown body IS the system prompt. Simple and self-documenting.
2. **OASF Records** (`.json`) — Structured JSON files following the Open Agentic Schema Framework standard.

Both formats let you tailor the agent's system prompt, permissions, model, and behaviour without changing any code.

Table of Contents

- Quick Start — Profiles (.md)
- Quick Start — OASF (.json)
- Discovery Paths
- Profile Format (.md)
- OASF Schema Reference (.json)
- Template Variables
- Permission Rules
- Validation Rules
- Persistent Memory
- Examples
- Using Profiles in Team Blueprints
- Built-in agent presets
- Slash Commands

Quick Start — Profiles (.md)

The easiest way to create a custom agent. Write a markdown file — the body becomes the system prompt.

1. Create the agents directory:

   ```
   mkdir -p .ragent/agents
   ```

2. Create a profile:

   ```
   cat > .ragent/agents/my-agent.md << 'EOF'
   ---
   {
     "name": "my-agent",
     "description": "A helpful assistant for my project"
   }
   ---

   You are a helpful AI assistant working on this project.

   Focus on clear, concise answers. When editing code, follow the existing
   ```

```
style and conventions.
EOF
```

3. Start ragent — your agent loads automatically. Use `/agents` to verify.

Quick Start — OASF (.json)

For full OASF compatibility or when you need the structured envelope format.

1. Create the agents directory:

```
# User-global (available in every project)
mkdir -p ~/.ragent/agents

# Project-local (this project only, takes priority)
mkdir -p .ragent/agents
```

2. Copy an example and customise it:

```
cp examples/agents/minimal-agent.json ~/.ragent/agents/my-agent.json
$EDITOR ~/.ragent/agents/my-agent.json
```

3. Start ragent — your agent loads automatically. Use `/agents` to verify it appeared, or `/agent` to pick it from the interactive list.

Discovery Paths

ragent searches two directories at startup:

Priority	Directory	Scope
Lower	`~/.ragent/agents/`	All projects (user-global)
Higher	`[PROJECT]/.ragent/agents/`	This project only

The **project directory** is the nearest ancestor of the current working directory that contains a `.ragent/agents/` subdirectory. Project-local definitions override user-global definitions when both have the same `name`.

Subdirectories are searched recursively. Both `.md` (profile) and `.json` (OASF) files are loaded.

Profile Format (.md)

A profile is a markdown file with a JSON frontmatter block between `---` delimiters. Everything after the closing `---` becomes the `system_prompt`.

Structure

```
---
{
  "name": "agent-name",
  "description": "One-line summary"
}
---

Your system prompt goes here. This is the markdown body.

It can contain **rich formatting**, code blocks, lists - anything the
model can interpret as instructions.
```

Frontmatter Fields

Field	Type	Default	Description
`name`	string	—	**Required.** Unique agent identifier (kebab-case, no spaces).
`description`	string	—	**Required.** One-line summary shown in `/agents` and the picker.
`mode`	string	`"all"`	Availability: `"primary"`, `"subagent"`, or `"all"`.
`model`	string	*inherited*	Lock to a specific model: `"provider:model"` or `"provider:model@vendor"` format (e.g. `"anthropic:claude-sonnet-4-20250514"`). When omitted the agent inherits the globally-selected model from `/provider`.
`max_steps`	integer	`1024`	Maximum tool-call steps before the agent stops.
`temperature`	float	provider default	Sampling temperature in `[0.0, 2.0]`.
`top_p`	float	provider default	Nucleus sampling probability in `[0.0, 1.0]`.
`hidden`	bool	`false`	When `true`, hidden from picker but available via `/agent <name>`.
`memory`	string	`"none"`	Persistent memory scope: `"none"`, `"user"`, or `"project"`. See Persistent Memory.
`permissions`	object[]	default ruleset	Permission rules.
`skills`	string[]	`[]`	Skill names preloaded for this agent.
`options`	object	`{}`	Provider-specific options passed through verbatim.

The markdown body supports template variables just like the OASF `system_prompt` field.

OASF Schema Reference (.json)

Each custom agent is a JSON file containing one OASF record. The top-level fields follow the OASF envelope; ragent-specific configuration lives inside the `ragent/agent/v1` module payload.

Top-Level Fields

Field	Type	Required	Description
`name`	string		Unique agent identifier. No spaces. Used as the agent selector key.
`description`	string		One-line summary shown in `/agents` and the picker.
`version`	string		Semantic version of this agent definition (required for `.json` records; markdown profiles synthesise it).
`schema_version`	string		OASF schema version (e.g. `"0.7.0"`; required for `.json` records).
`authors`	string[]		List of author names or email addresses.
`created_at`	string		ISO 8601 creation timestamp.
`skills`	object[]		OASF skill taxonomy annotations (informational only).
`domains`	object[]		OASF domain taxonomy annotations (informational only).
`locators`	object[]		Source/artifact locators (informational only).
`modules`	object[]		Must contain at least one entry with `"type": "ragent/agent/v1"`.

`ragent/agent/v1` Payload Fields

Field	Type	Default	Description
`system_prompt`	string	—	**Required.** The agent's system prompt. Supports template variables. Max 32,768 chars.
`mode`	string	`"all"`	Availability: `"primary"` (user-selectable), `"subagent"` (delegation only), `"all"` (both).

Field	Type	Default	Description
`max_steps`	integer	`1024`	Maximum tool-call steps before the agent stops. Must be ≥ 1.
`temperature`	float	provider default	Sampling temperature in `[0.0, 2.0]`.
`top_p`	float	provider default	Nucleus sampling probability in `[0.0, 1.0]`.
`model`	string	*inherited*	Lock to a specific model: `"provider:model"` or `"provider:model@vendor"` format (e.g. `"anthropic:claude-opus-4-5"`, `"openai:gpt-4o@azure"`). When omitted the agent inherits the globally-selected model.
`hidden`	bool	`false`	When `true`, the agent is available for direct switch (`/agent <name>`) but not shown in the picker or `/agents` list.
`memory`	string	`"none"`	Persistent memory scope: `"none"`, `"user"`, or `"project"`. See Persistent Memory.
`permissions`	object[]	default ruleset	Permission rules. Omit to inherit the default ruleset.
`options`	object	`{}`	Provider-specific options passed through verbatim (e.g. `{"max_tokens": 4096}`).
`skills`	string[]	`[]`	Ragent skill names preloaded for this agent (e.g. `"simplify"`).
`thinking`	object	*inherit*	Extended-thinking config (`{"enabled": true, "level": "high", "budget_tokens": 16000}`). Omit to inherit the provider/model default.

Template Variables

The following placeholders in `system_prompt` are substituted at session start:

Variable	Replaced With
`{{WORKING_DIR}}`	Absolute path of the current working directory
`{{FILE_TREE}}`	Two-level directory listing of the working directory
`{{AGENTS_MD}}`	Contents of `AGENTS.md` in the project root (if it exists)
`{{GIT_STATUS}}`	Output of `git status` for the working directory
`{{README}}`	Contents of the project `README.md` (if it exists)

Variable	Replaced With
`{{DATE}}`	Current date in `YYYY-MM-DD` format (UTC)

Sections whose content was already embedded via a template variable are not auto-appended by the agent system, so there is no duplication.

Example

```
"system_prompt": "You are a documentation writer.\nProject: {{WORKING_DIR}}\nDate: {{DATE}}\n\n{{
```

Permission Rules

The `permissions` array controls what file and shell operations the agent may perform without asking for confirmation. Rules are evaluated in order; the first match wins.

Rule Object

```
{ "permission": "<category>", "pattern": "<glob>", "action": "<action>" }
```

Field	Values	Description
`permission`	`read`, `edit` (alias `write`), `bash` (alias `execute`), `web` (alias `fetch`), `task`, `plan_enter` (alias `plan`), `plan_exit`	Operation category. `question` is legacy-only; namespaced forms (e.g. `file:read`) are normalised to the base type; unknown names are accepted as custom permissions.
`pattern`	glob string	Files or commands the rule matches (e.g. `"**"`, `"src/**/*.rs"`)
`action`	`allow`, `deny`, `ask`	What to do when matched

Example — Read-Only Agent

```
"permissions": [
  { "permission": "read", "pattern": "**", "action": "allow" },
  { "permission": "edit", "pattern": "**", "action": "deny"  },
  { "permission": "bash", "pattern": "**", "action": "deny"  }
]
```

Example — Docs-Only Writer

```
"permissions": [
  { "permission": "read",  "pattern": "**",      "action": "allow" },
  { "permission": "edit",  "pattern": "docs/**", "action": "allow" },
  { "permission": "edit",  "pattern": "**/*.md", "action": "allow" },
  { "permission": "edit",  "pattern": "**",      "action": "ask"   },
  { "permission": "bash",  "pattern": "**",      "action": "deny"  }
]
```

Validation Rules

If a file fails validation it is skipped with a non-fatal diagnostic (shown at startup in the log panel and in `/agents → Diagnostics`).

Common Rules (both formats)

Condition	Error
`name` is empty or contains spaces	`agent name must be non-empty and contain no spaces`
`description` is empty	`description must not be empty`
`system_prompt` is empty	`system_prompt must not be empty`
`system_prompt` exceeds 32,768 chars	`system_prompt too long (N chars; max 32768)`
`mode` is unrecognised	`unknown mode '<value>'; expected primary, subagent, or all`
`temperature` outside `[0.0, 2.0]`	`temperature N out of range [0.0, 2.0]`
`top_p` outside `[0.0, 1.0]`	`top_p N out of range [0.0, 1.0]`
`model` not in `provider:model` format	`model '<value>' must be in 'provider:model' or 'provider:model@vendor' format`
`memory` not one of `none`, `user`, `project`	`unknown memory scope '<value>'; expected user, project, or none`
`max_steps` is 0	`max_steps must be greater than 0`
Permission `action` unrecognised	`unknown action '<value>'; expected allow, deny, or ask`
`name` collides with a built-in	Warning (not skip): loaded as `custom:<name>`

Profile-Specific Rules (.md)

Condition	Error
Missing `---` frontmatter delimiters	`missing JSON frontmatter (expected --- delimiters)`
Invalid JSON in frontmatter	`frontmatter JSON parse error: ...`
Empty markdown body after `---`	`markdown body (system_prompt) must not be empty`

OASF-Specific Rules (.json)

Condition	Error
No `ragent/agent/v1` module	`missing required module type 'ragent/agent/v1'`

Persistent Memory

Agents can maintain a persistent memory directory that survives across sessions. This lets teammates build institutional knowledge — learned patterns, project conventions, frequently needed context — that is automatically injected into their system prompt at spawn time.

Memory Scopes

Scope	Directory	Use Case
`"none"`	*(disabled)*	Default. No memory directory.
`"user"`	`~/.ragent/agent-memory/<agent-name>/`	User-global memory shared across all projects.
`"project"`	`.ragent/agent-memory/<agent-name>/`	Project-local memory specific to the repository.

How It Works

1. **At spawn** — If memory is enabled, ragent reads `MEMORY.md` from the agent's memory directory and injects the first 200 lines (or 25 KB) into the system prompt.
2. **During execution** — The agent can read/write files in its memory directory using the `team_memory_read` and `team_memory_write` tools.
3. **Across sessions** — The memory directory persists on disk, so information written in one session is available in the next.

Memory Tools

Tool	Description
`team_memory_read`	Read a file from the agent's memory directory. Defaults to `MEMORY.md`.
`team_memory_write`	Write or append to a file in the memory directory. Defaults to append mode on `MEMORY.md`.

Example — Agent with Project Memory

```
---
{
  "name": "architect",
```

```
  "description": "System architect with project memory",
  "memory": "project"
}
---

You are a system architect. Review code structure, suggest improvements,
and document architectural decisions.

Use your memory to record:
- Key architectural decisions and rationale
- Component relationships and dependencies
- Known technical debt items
```

Memory Scope Inheritance

When a teammate is spawned in a team:

1. Blueprint `memory` field (from `spawn-prompts.json`) takes priority
2. Falls back to the agent profile's `memory` field
3. Falls back to `"none"` (disabled)

Examples

Minimal Profile (.md)

```
---
{
  "name": "my-agent",
  "description": "A minimal custom agent"
}
---

You are a helpful AI agent.
Working directory: {{WORKING_DIR}}

{{AGENTS_MD}}
```

Security Reviewer Profile (.md)

```
---
{
  "name": "security-reviewer",
  "description": "OWASP-focused security code reviewer",
  "mode": "subagent",
  "max_steps": 30,
  "temperature": 0.2,
  "permissions": [
    { "permission": "read", "pattern": "**", "action": "allow" },
    { "permission": "edit", "pattern": "**", "action": "deny"  },
```

```
    { "permission": "bash", "pattern": "**", "action": "deny"  }
  ]
}
---

You are a security-focused code reviewer specialising in the OWASP Top 10.

For every review:
1. Identify injection flaws (SQL, command, LDAP, XPath)
2. Check authentication and session management weaknesses
3. Look for sensitive data exposure (keys, tokens, PII in logs)
4. Verify access controls and authorisation logic
5. Check for security misconfigurations

Report only high-signal issues with file paths and concrete fixes.
```

Minimal OASF Agent (.json)

```
{
  "name": "my-agent",
  "description": "A minimal custom agent example",
  "version": "1.0.0",
  "schema_version": "0.7.0",
  "modules": [{
    "type": "ragent/agent/v1",
    "payload": {
      "system_prompt": "You are a helpful AI agent.\nWorking directory: {{WORKING_DIR}}\n\n{{AGEN
      "mode": "primary",
              "max_steps": 1024    }
  }]
}
```

Security Reviewer OASF (.json)

Read-only OWASP-focused reviewer (see `examples/agents/security-reviewer.json`):

```
{
  "name": "security-reviewer",
  "description": "OWASP-focused security code reviewer",
  "version": "1.0.0",
  "schema_version": "0.7.0",
  "modules": [{
    "type": "ragent/agent/v1",
    "payload": {
      "system_prompt": "You are a security-focused code reviewer...\n{{WORKING_DIR}}\n{{AGENTS_MD
      "mode": "primary",
      "max_steps": 30,
      "temperature": 0.2,
      "permissions": [
        { "permission": "read", "pattern": "**", "action": "allow" },
```

```
        { "permission": "edit", "pattern": "**", "action": "deny"  },
        { "permission": "bash", "pattern": "**", "action": "deny"  }
      ]
    }
  }]
}
```

See `examples/agents/` for complete, ready-to-use agent files.

Using Profiles in Team Blueprints

Team blueprints can reference agent profiles by name in `spawn-prompts.json` using either the `agent_type` or `profile` key:

```
[
  {
    "tool_name": "team_spawn",
    "teammate_name": "reviewer",
    "profile": "security-reviewer",
    "prompt": "Review the authentication module for vulnerabilities."
  }
]
```

When ragent spawns the teammate, it resolves `"security-reviewer"` via the same discovery pipeline — loading the `.md` or `.json` agent definition from `.ragent/agents/`. The profile's system prompt, permissions, model, and other settings are applied to the spawned teammate.

> **Tip:** `"profile"` is an alias for `"agent_type"`. Both work identically in `spawn-prompts.json`. Use `"profile"` when referencing a declarative agent profile for clarity.

Built-in agent presets

ragent ships with the following built-in agent presets. Custom agents loaded from `.ragent/agents/` or `~/.ragent/agents/` extend this list.

Each preset has a dedicated system prompt that shapes the agent's behaviour, expertise, and coding conventions. The prompts are defined in `crates/ragent-agent/src/agent/mod.rs` (`create_builtin_agents`).

> **Note:** The built-in agent list was significantly expanded in v1.0.40+ to include domain-specific specialists. All are available via `/agent <name>` or the interactive picker (`/agent`).

Primary agents (user-selectable)

Primary agents appear in the `/agent` interactive picker and can be selected with `/agent <name>`. They have full or read-only tool access depending on their role.

`ask` — **Quick Q&A** **System prompt:**

> You are a helpful AI assistant. Answer the user's questions clearly and concisely. You do not have access to any tools — just respond with your best knowledge.

Permissions: Read-only (no file writes, no shell). **Temperature:** Default. **Thinking:** Off.

This agent has no tools at all — it is a pure conversational assistant. Use it when you want a quick answer without the agent reading or modifying files.

Example:

```
/agent ask
Explain the difference between async/await and threads in Rust
```

`general` — **General-purpose coding agent (default)** **System prompt:**

> You are a powerful AI coding assistant. You help users with software development tasks including writing code, debugging, reviewing, and explaining code. You have access to tools for reading, writing, and editing files, executing shell commands, and searching codebases. Use 'grep' or 'search' to find text/code patterns, 'glob' to find files by name, 'list' to view directory contents, and 'read' to view file contents. Always prefer using tools to verify your assumptions rather than guessing.

Permissions: Full (read, write, shell, search).

This is the default agent. It has broad tool access and is suitable for most coding tasks.

Example:

```
/agent general
Find all uses of the deprecated `unwrap()` call in src/ and replace
them with proper error handling using `?`
```

`rust-coder` — **Rust coding specialist** **System prompt:**

> You are a Rust coding specialist. You write idiomatic, production-grade Rust code with an emphasis on zero-cost abstractions, memory safety, and composability.
>
> Expertise: - Ownership, borrowing, and lifetimes - Error handling with Result<T, E> and anyhow/thiserror - Async Rust with tokio and futures - Traits and trait objects (dyn Trait vs impl Trait) - Unsafe code when necessary (with safety comments) - Cargo workspace management and dependency hygiene - Testing with cargo test, mockall, and insta - Performance: zero-copy, SIMD, rayon parallelism
>
> When reviewing or writing Rust: - Prefer `?` over `.unwrap()` / `.expect()` in library code - Use `tracing` (not println!) for structured logging - Follow the Rust API Guidelines and naming conventions - Minimize allocations; prefer iterators over loops - Document public APIs with `///` doc comments

Permissions: Full. **Temperature:** Default.

Example:

```
/agent rust-coder
Refactor the parser module to use thiserror for error types instead
of anyhow, and add doc comments to all public functions
```

`python-coder` — **Python coding specialist** **System prompt:**

> You are a Python coding specialist. You write clean, idiomatic Python following modern best practices and PEP 8.
>
> Expertise: - Type hints (PEP 484), generics, and mypy/pyright compliance - Async Python with asyncio, aiohttp, and FastAPI - Data modelling with dataclasses, Pydantic, and attrs - Testing with pytest, unittest, and coverage - Packaging with pyproject.toml, poetry, and uv - Virtual environments and dependency management - Performance: profiling, caching (functools.lru_cache), vectorisation - Python 3.11+ features (task groups, exception groups, tomllib)
>
> When reviewing or writing Python: - Use type hints everywhere; avoid bare `Any` - Prefer f-strings and pathlib over os.path - Use context managers (`with`) for resource cleanup - Prefer composition over inheritance - Use `isinstance()` checks, not `type()` comparisons - Keep functions small and testable (single responsibility)

Permissions: Full. **Temperature:** Default.

Example:

```
/agent python-coder
Add type hints to all functions in src/handlers.py and fix any mypy
errors that come up
```

`typescript-coder` — **TypeScript/JavaScript coding specialist** **System prompt:**

> You are a TypeScript and JavaScript coding specialist. You write type-safe, modern JavaScript for both frontend and backend contexts.
>
> Expertise: - Strict TypeScript with explicit types; minimal use of `any` - Union types, discriminated unions, and type narrowing - Generic constraints and mapped types - Async/await, Promises, and error handling patterns - React hooks, Next.js, and component architecture - Node.js, Express, and Fastify server patterns - Testing with Vitest, Jest, and Playwright - Build tools: Vite, Rollup, Webpack, esbuild, tsup - Package managers: npm, pnpm, yarn (Berry)
>
> When reviewing or writing TS/JS: - Use `const` and `let`; avoid `var` - Prefer arrow functions for callbacks; named functions for hoisting - Use optional chaining (`?.`) and nullish coalescing (`??`) - Keep components small and focused; extract hooks early - Use ESLint + Prettier for consistency

Permissions: Full. **Temperature:** Default.

Example:

```
/agent typescript-coder
Convert the Express route handlers in src/routes/ to use proper
TypeScript types and add input validation with zod
```

`fastapi-agent` — **FastAPI backend specialist** **System prompt:**

> You are a FastAPI and Python web-backend specialist. You design and build high-performance REST and WebSocket APIs.
>
> Expertise: - FastAPI routing, dependency injection, and lifespan events - Pydantic v2 models, validators, and serialization - SQLAlchemy 2.0 ORM, Alembic migrations, and async engines - Authentication: OAuth2, JWT, API keys, and session management - Background tasks, Celery, and message queues - Docker multi-stage builds and docker-compose orchestration - Testing: pytest-asyncio, httpx.AsyncClient, TestClient - Deployment: Gunicorn + Uvicorn, ASGI servers, reverse proxies
>
> When designing APIs: - Use HTTP status codes correctly (201 Created, 204 No Content) - Version URLs (`/api/v1/...`) and use HATEOAS sparingly - Document all endpoints with OpenAPI (auto-generated by FastAPI) - Implement rate limiting and input validation at the edge - Use structured logging (JSON) for observability

Permissions: Full. **Temperature:** Default.

Example:

```
/agent fastapi-agent
Create a new POST /api/v1/users endpoint that accepts a Pydantic
model, validates the email, hashes the password, and returns 201
with the created user
```

`security-auditor` — **Security code reviewer** **System prompt:**

> You are a security-focused code reviewer specialising in the OWASP Top 10.
>
> For every review: 1. Identify injection flaws (SQL, command, LDAP, XPath, template) 2. Check authentication and session management weaknesses 3. Look for sensitive data exposure (keys, tokens, PII in logs) 4. Flag insecure direct object references and broken access control 5. Detect security misconfiguration and outdated dependencies 6. Highlight XXE and deserialization risks 7. Note XSS vectors, CSP bypasses, and CSRF weaknesses 8. Flag use of components with known vulnerabilities (CVE checks) 9. Check for insufficient logging and monitoring gaps
>
> Provide CWE identifiers and OWASP references for every finding. Suggest concrete mitigations with code examples.

Permissions: Read-only. **Temperature:** 0.2 (low, for deterministic findings).

Example:

```
/agent security-auditor
Review the authentication module in src/auth/ for OWASP Top 10
vulnerabilities and provide CWE references for each finding
```

`test-writer` — **Test generation specialist** **System prompt:**

> You are a test-writing specialist. You generate comprehensive test suites that verify behaviour, not just achieve coverage numbers.
>
> Expertise: - Unit tests: arrange-act-assert, table-driven tests, property-based testing - Integration tests: database fixtures, HTTP client tests, API contracts - E2E tests: Playwright, Cypress, user-journey scenarios - Mocking and stubbing (mockall, Mockito, jest.mock, sinon) - Coverage analysis: branch coverage, mutation testing - CI-friendly tests: idempotent, parallel-safe, deterministic
>
> When writing tests: - Test one thing per function; use descriptive names - Test edge cases, error paths, and boundary conditions - Use fixtures and factories for test data, not hard-coded values - Mock at the boundary; test real collaborators where possible - Keep tests fast (< 100ms per test ideally) - Add `#[should_panic]` / `pytest.raises` for expected failures

Permissions: Full. **Temperature:** 0.3 (low, for deterministic test generation).

Example:

```
/agent test-writer
Write unit tests for the parse_config function in src/config.rs.
Include edge cases for empty input, invalid JSON, and missing fields
```

`documenter` — **Documentation specialist** **System prompt:**

> You are a technical documentation specialist. You write clear, concise documentation that helps developers understand and use code.
>
> Expertise: - API documentation: docstrings, OpenAPI specs, type signatures - README files: quick-start, installation, configuration, examples - Architecture Decision Records (ADRs) and design docs - User guides and tutorials with runnable examples - Changelog management (Keep a Changelog format) - Inline comments for complex algorithms and business logic
>
> When documenting: - Lead with the "why", then the "what", then the "how" - Include practical code examples that compile/run - Use tables for parameter references and configuration options - Keep headings hierarchical and scannable - Cross-reference related documents with relative links - Update tables of contents when adding new sections

Permissions: Full. **Temperature:** 0.5 (moderate, for natural prose).

Example:

```
/agent documenter
Generate a README.md for the crates/ragent-research crate that
includes installation, quick start, configuration, and architecture
overview
```

`devops-agent` — **DevOps and infrastructure specialist** **System prompt:**

> You are a DevOps and infrastructure specialist. You design, build, and maintain deployment pipelines and cloud infrastructure.
>
> Expertise: - Containerisation: Docker, BuildKit, multi-stage builds, distroless images - Orchestration: Kubernetes manifests, Helm charts, Kustomize - CI/CD: GitHub Actions, GitLab CI, Azure DevOps, ArgoCD - Infrastructure as Code: Terraform, Pulumi, AWS CDK, CloudFormation - Monitoring: Prometheus, Grafana, OpenTelemetry, structured logging - Secrets management: Vault, Sealed Secrets, AWS Secrets Manager - Networking: Ingress, service mesh (Istio, Linkerd), TLS termination - Cloud platforms: AWS, GCP, Azure (serverless, VMs, managed services)
>
> When working on infrastructure: - Use declarative configuration (YAML, HCL) over imperative scripts - Implement health checks, readiness probes, and graceful shutdowns - Follow the principle of least privilege for IAM and RBAC - Version-pin all base images and dependencies - Document runbooks and rollback procedures

Permissions: Full. **Temperature:** Default.

Example:

```
/agent devops-agent
Create a GitHub Actions workflow that builds the Rust binary, runs
tests, and publishes a Docker image to GHCR on tag pushes
```

`database-agent` — **Database specialist** **System prompt:**

> You are a database specialist. You design schemas, write queries, and optimise data access patterns for relational and NoSQL databases.
>
> Expertise: - Relational: PostgreSQL, MySQL, SQLite — schema design, indexing, query plans - NoSQL: MongoDB, Redis, DynamoDB — document modelling, key patterns - Migrations: Alembic, Flyway, dbmate — forward-only, rollback-safe - ORMs: SQLAlchemy, Diesel, Prisma, TypeORM — type-safe query builders - Performance: EXPLAIN ANALYZE, query rewriting, materialised views - Transactions: ACID guarantees, isolation levels, deadlock avoidance - Data integrity: constraints, triggers, foreign keys, normalisation - Backup and replication: pg_dump, logical replication, read replicas
>
> When working with databases: - Normalise to 3NF initially; denormalise selectively for read performance - Add indexes after profiling; avoid over-indexing on write-heavy tables - Use connection pooling (PgBouncer, r2d2, sqlx::Pool) - Parameterise queries; never concatenate user input into SQL - Add database-level constraints as a safety net, not just application validation

Permissions: Full. **Temperature:** Default.

Example:

```
/agent database-agent
Design a PostgreSQL schema for a multi-tenant SaaS application with
separate tenant isolation, and write the initial Alembic migration
```

`frontend-agent` — **Frontend development specialist** **System prompt:**

> You are a frontend web development specialist. You build responsive, accessible, and performant user interfaces.
>
> Expertise: - React: hooks, context, suspense, server components, Next.js App Router - Vue: Composition API, Pinia, Nuxt.js, VueUse - Styling: Tailwind CSS, CSS-in-JS (styled-components, emotion), PostCSS - State management: Zustand, Redux Toolkit, Pinia, signals (Solid, Preact) - Accessibility: ARIA roles, keyboard navigation, focus management, axe - Performance: Core Web Vitals, code splitting, image optimisation, caching - Testing: React Testing Library, Vitest, Playwright, Storybook - Build tools: Vite, Webpack, esbuild, SWC, Turbopack
>
> When building frontend: - Mobile-first responsive design with Tailwind breakpoints - Ensure WCAG 2.1 AA compliance (contrast ratios, focus indicators) - Use semantic HTML (`<header>`, `<nav>`, `<main>`, `<article>`) - Lazy-load images and heavy components below the fold - Keep bundle sizes small; tree-shake unused dependencies - Use `key` props correctly in lists; avoid index-as-key

Permissions: Full. **Temperature:** Default.

Example:

```
/agent frontend-agent
Build a responsive React navbar component with Tailwind CSS that
collapses to a hamburger menu on mobile, with keyboard-accessible
focus management
```

Sub-agents (spawned programmatically)

Sub-agents are not user-selectable via `/agent`. They are spawned internally by the agent loop (via `new_agent`) or by TUI features like session title generation. However, they can be used directly by the model when it delegates work.

`build` — **Build and test agent** **System prompt:**

> You are a build agent specializing in compiling, testing, and debugging software projects. Focus on running builds, fixing compilation errors, running tests, and ensuring code quality. Use bash commands to interact with build systems and test frameworks.

Permissions: Full. **Hidden:** No.

Use this agent when you need to compile, run tests, or fix build breakages. It has full tool access including shell execution.

Example:

```
Run the test suite for the ragent-codeindex crate and fix any
compilation errors that come up
```

The model would spawn this as:

```
{"agent": "build", "task": "Run cargo test -p ragent-codeindex and fix any compilation errors"}
```

`plan` — **Planning agent** **System prompt:**

> You are a planning agent. Your job is to analyze requirements and create detailed implementation plans. Read the codebase to understand existing patterns and architecture. Output a structured plan with clear steps. Do NOT make any changes yourself — only plan and document.

Permissions: Read-only. **Temperature:** 0.7 (higher, for creative planning). **Hidden:** No.

Use this agent when you need an implementation plan without making changes. It reads the codebase and produces structured plans.

Example:

```
Create an implementation plan for adding webhook support to the
existing notification system
```

The model would spawn this as:

```
{"agent": "plan", "task": "Analyze the notification system in src/notify/ and create a plan for a
```

`explore` — **Exploration agent** **System prompt:**

> You are an exploration agent specializing in understanding codebases. Use read, grep, glob, and list tools to navigate and understand code. Provide concise, accurate answers about code structure, patterns, and logic. Do NOT modify any files.

Permissions: Read-only. **Hidden:** No.

Use this agent for any codebase search, reading, or understanding task. It is the fastest and cheapest agent and should be preferred for exploration.

Example:

```
Find all callers of the process_message function and explain how
messages flow through the system
```

The model would spawn this as:

```
{"agent": "explore", "task": "Find all callers of process_message in src/ and explain the message
```

`title` — **Session title generator (hidden)** **System prompt:**

> Generate a short, descriptive title (3-6 words) for a coding session based on the conversation. Output ONLY the title, nothing else.

Permissions: None. **Temperature:** 0.3 (low, for consistency). **Hidden:** Yes.

This agent is spawned automatically by the TUI to generate a session title. It is not user-facing.

Example (internal):

The TUI spawns this agent after the first user message:

```
{"agent": "title", "task": "Generate a session title for: \"Add OAuth2 support to the Gmail tool\
```

`summary` — Session summarizer (hidden) **System prompt:**

> Summarize the conversation so far into a concise paragraph that captures the key topics discussed, decisions made, and work completed.

Permissions: None. **Temperature:** 0.3 (low, for accuracy). **Hidden:** Yes.

This agent is spawned automatically during context compaction to summarize earlier conversation turns. It is not user-facing.

Example (internal):

The session loop spawns this during compaction:

```
{"agent": "summary", "task": "Summarize the conversation so far: <compacted messages>"}
```

Permission and temperature summary

Agent	Mode	Permissions	Temperature	Hidden
`ask`	Primary	Read-only	Default	No
`general`	Primary	Full	Default	No
`rust-coder`	Primary	Full	Default	No
`python-coder`	Primary	Full	Default	No
`typescript-coder`	Primary	Full	Default	No
`fastapi-agent`	Primary	Full	Default	No
`security-auditor`	Primary	Read-only	0.2	No
`test-writer`	Primary	Full	0.3	No
`documenter`	Primary	Full	0.5	No
`devops-agent`	Primary	Full	Default	No
`database-agent`	Primary	Full	Default	No
`frontend-agent`	Primary	Full	Default	No
`build`	Subagent	Full	Default	No
`plan`	Subagent	Read-only	0.7	No
`explore`	Subagent	Read-only	Default	No
`title`	Subagent	None	0.3	Yes
`summary`	Subagent	None	0.3	Yes

Slash Commands

Command	Description
`/agents`	List all agents (built-in and custom) with scope, format, and diagnostics
`/agent`	Open interactive picker — custom agents show a yellow `[custom]` badge

Command	Description
`/agent <name>`	Switch directly to a named agent

Custom agents loaded from the project directory show `[project/profile]` or `[project/oasf]` scope in `/agents`; user-global agents show `[global/profile]` or `[global/oasf]`.